

Keith Ando Ogawa

Seleção de Camadas para Ajuste Fino Eficiente de Grandes Modelos de Linguagem

São Paulo, SP

2025

Keith Ando Ogawa

Seleção de Camadas para Ajuste Fino Eficiente de Grandes Modelos de Linguagem

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Artur Jordão Lima Correia

São Paulo, SP

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

Ogawa, Keith Ando

Seleção de camadas para ajuste fino eficiente de grandes modelos de linguagem / K. A. Ogawa -- São Paulo, 2025.

43 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.Redes Neurais 2.Aprendizagem Profunda 3.Aprendizado de Máquina
4.Processamento de Linguagem Natural I.Universidade de São Paulo. Escola Politécnica. Departamento de Engenharia de Computação e Sistemas Digitais II.t.

*Este trabalho é dedicado às crianças adultas que,
quando pequenas, sonharam em se tornar cientistas.*

Agradecimentos

Agradeço à minha família, pelo amor, apoio e confiança. Em especial, aos meus pais, Filemon e Helena, que nunca mediram esforços para me oferecer suporte ao longo de toda a minha formação.

Expresso também minha gratidão ao meu orientador, Prof. Dr. Artur Jordão, pelas contribuições, direcionamentos e ensinamentos, não apenas neste trabalho, mas também em minha formação como pesquisador.

Por fim, este trabalho foi realizado com apoio do Itaú Unibanco S.A, por meio do Programa de Bolsas Itaú (PBI).

Resumo

Grandes modelos de linguagem pré-treinados em corpos de dados massivos tornaram-se fundamentais para o avanço da inteligência artificial de propósito geral. Um paradigma popular para melhorar suas habilidades em uma tarefa específica envolve adaptar seu conhecimento através do ajuste fino em uma base de dados reduzida. Apesar da quantidade de dados inferior ao treino inicial, este ajuste torna-se custoso à medida que os modelos crescem em quantidade de parâmetros. Para tratar deste problema, métodos eficientes de ajuste fino, como *Low-Rank Adaptation* (LoRA), propõem o ajuste em uma parcela menor de parâmetros e alcançam desempenho comparável ao do ajuste completo. Neste trabalho, buscamos complementar estes métodos através da redução de parâmetros por uma direção ortogonal à intra-camada normalmente abordada: a seleção de camadas para o ajuste. Acreditamos que as camadas dos grandes modelos contribuem de forma desigual para uma tarefa específica. Com base nisso, escolhemos camadas que geram grandes mudanças nas representações internas do modelo para esta etapa de treino. Com uma técnica de integração imediata com LoRA e suas variantes, alcançamos quedas desprezíveis e até melhoras de desempenho preditivo mesmo com reduções drásticas na quantidade de parâmetros. Do ponto de vista de eficiência, nosso método promove uma aceleração no processo de treino, o que se traduz em menor demanda por infraestrutura computacional, bem como em redução do consumo energético.

Palavras-chave: Ajuste Fino. Grandes Modelos de Linguagem. Aprendizado de Máquina. Inteligência Artificial Sustentável.

Abstract

Pre-training large language models on a massive corpus of data becomes fundamental for advancing general-purpose artificial intelligence. A popular paradigm to enhance their abilities in a specific downstream task typically involves adapting their acquired knowledge through fine-tuning with a smaller dataset. Despite the smaller amount of data in comparison with the pre-training, the fine-tuning process becomes expensive as models grow in parameter size. To address this problem, parameter-efficient fine-tuning methods, such as Low-Rank Adaptation (LoRA), adjust a small number of parameters and achieve a predictive performance comparable to full fine-tuning. In this work, we aim to complement these methods by reducing the number of parameters through a direction orthogonal to intra-layer approaches: selecting a subset of layers to fine-tune. We argue that not all layers contribute equally to the model adaptation for a specific downstream task. Building upon this, we identify layers that contribute the most to changes in the internal representations. Leveraging a technique readily compatible with LoRA and its variants, we achieve negligible drops or even improvements in predictive performance, even with drastic reductions in the number of trainable parameters. From the efficiency perspective, our method promotes an acceleration in the training process, which translates into less demand for computational infrastructure and reductions in energy consumption.

Keywords: Fine tuning. Large Language Models. Machine Learning. Sustainable Artificial Intelligence.

Lista de ilustrações

Figura 1 – Desempenho do modelo DeBERTa-v3 _{base} com diferentes camadas. . . .	14
Figura 2 – Arquitetura <i>transformer</i> Encoder-Decoder	16
Figura 3 – Arquitetura do Modelo LLaVA	20
Figura 4 – Visão Geral do Nosso Método	24
Figura 5 – Compilado de resultados dos modelos RoBERTa _{base} e DeBERTa-v3 _{base}	27
Figura 6 – Heurísticas de seleção de camadas para o ajuste-fino.	29
Figura 7 – Aceleração em treino do modelo RoBERTa _{base}	33
Figura 8 – Memória alocada e reservada da GPU do modelo RoBERTa _{base}	34
Figura 9 – Memória alocada e reservada da GPU do modelo LLaMA-2-7B.	35
Figura 10 – Valor do CKA por camada do modelo RoBERTa _{base}	43

Lista de tabelas

Tabela 1	– Resultados dos modelos RoBERTa _{base} e DeBERTa-v3 _{base}	26
Tabela 2	– Resultados dos modelos LLaMA 2-7B, Mistral-7B-v0.1 and Gemma-7B.	28
Tabela 3	– Resultados do modelo RoBERTa _{base} com diferentes configurações de camadas selecionadas.	29
Tabela 4	– Resultados dos modelos LLaMA 2-7B, Mistral-7B-v0.1 and Gemma-7B com seleção das camadas centrais.	30
Tabela 5	– Resultados do modelo RoBERTa _{base} com seleção de diferentes quantidades de camadas.	31
Tabela 6	– Resultados do modelo RoBERTa _{base} com aumento de <i>rank</i> nas camadas selecionadas.	32

Lista de abreviaturas e siglas

CKA	<i>Centered Kernel Alignment</i>
FIM	<i>Fisher Information Matrix</i>
GB	<i>Gigabyte</i>
GPU	<i>Graphics Processing Unit</i>
HSIC	<i>Hilbert-Schmidt Independence Criterion</i>
LoRA	<i>Low-Rank Adaptation</i>
MB	<i>Megabyte</i>
MLP	<i>Multilayer Perceptron</i>
PEFT	<i>Parameter-Efficient Fine-tuning</i>
PiSSA	<i>Principal Singular Values and Singular Vectors Adaptation</i>
ReLU	<i>Rectified Linear Unit</i>
RNN	<i>Recurrent Neural Networks</i>
SVD	<i>Singular Value Decomposition</i>

Sumário

1	INTRODUÇÃO	12
1.1	Motivação	12
1.2	Objetivos	13
1.3	Justificativa	13
1.4	Definição do Problema Abordado	14
2	ASPECTOS CONCEITUAIS	15
2.1	Arquitetura <i>transformer</i>	15
2.1.1	Arquitetura <i>transformer</i> encoder-decoder	15
2.1.1.1	Camada de <i>embedding</i> e Tokenizador	15
2.1.1.2	Codificador Posicional	17
2.1.1.3	Módulos	17
2.1.1.3.1	Módulo de atenção	17
2.1.1.3.2	Módulo <i>Multilayer Perceptron</i> (MLP)	18
2.1.2	Arquitetura <i>transformer encoder-only</i>	18
2.1.3	Arquitetura <i>transformer decoder-only</i>	19
2.1.4	Arquiteturas multimodais	19
2.2	<i>Parameter Efficient Fine-tuning</i> (PEFT)	19
2.2.1	<i>Low-Rank Adaptation</i> (LoRA)	20
2.2.2	<i>Principal Singular Values and Singular Vectors Adaptation</i> (PiSSA)	21
2.3	Métricas de similaridade de representação	21
2.3.1	<i>Centered Kernel Alignment</i> (CKA)	21
2.4	Seleção de Camadas para Ajuste Fino	22
2.4.1	Seleção de Camadas para modelos <i>encoder-only</i>	22
3	MÉTODO PROPOSTO	23
3.1	Método proposto	23
4	EXPERIMENTOS	25
4.1	Tecnologias utilizadas	25
4.1.1	Python	25
4.1.2	PyTorch e bibliotecas do <i>HuggingFace</i>	25
4.2	Avaliação em compreensão de texto	25
4.3	Avaliação em geração de texto	27
4.4	Comparação com heurísticas simples	28
4.5	Quanto ao número de blocos selecionados	31

4.6	Quanto ao papel do <i>rank</i>	31
4.7	Quanto à eficiência computacional	32
4.8	Efetividade em modelos multimodais	35
5	CONSIDERAÇÕES FINAIS	37
5.1	Conclusões	37
5.2	Perspectivas de continuidade	38
	REFERÊNCIAS	39
	APÊNDICES	42
	APÊNDICE A – A PRIMEIRA CAMADA DE MODELOS <i>ENCODER-ONLY</i>	43

1 Introdução

1.1 Motivação

Proposta inicialmente para realizar tarefas de tradução, a arquitetura *transformer* (VASWANI et al., 2017) redefiniu o estado da arte em diversas áreas da inteligência artificial, como processamento de linguagem natural e visão computacional (MASLEJ, 2025; BENGIO; AL., 2025). A sua estrutura centrada no mecanismo de atenção permite a incorporação de informação relativa à atenção de um trecho de uma sentença analisada em relação aos outros, o que possibilita um rico entendimento da entrada como um todo. Além disso, em contraste com o estado da arte anterior baseado em redes neurais recorrentes (RNNs, do inglês *Recurrent Neural Networks*), os modelos baseados na arquitetura *transformer* têm a capacidade de realizar o treinamento de forma paralela. Tal paralelismo desempenha uma função fundamental para o aumento de escala para redes de bilhões de parâmetros que se mostram muito efetivas, como o GPT (OPENAI, 2023) e o DeepSeek (DEEPSEEK-AI et al., 2024).

Acompanhando a maior capacidade preditiva destes grandes modelos, o custo computacional do seu treino e ajuste para tarefas específicas também aumentou. Por exemplo, um ajuste fino de um modelo LLaMA (TOUVRON et al., 2023) de 65 bilhões de parâmetros com precisão dos pesos reduzida para 16 bits requer mais de 780 *gigabytes* (GB) de memória na unidade de processamento gráfico (GPU, do inglês *Graphics Processing Unit*) (DETTMERS et al., 2023). Tal demanda computacional traduz-se em um desafio para a viabilidade do uso sustentável e financeiramente acessível dessa tecnologia.

Para lidar com os desafios acima, os métodos de ajuste-fino eficiente (PEFT, do inglês *Parameter-Efficient Fine-tuning*) surgem como uma solução para realizar o ajuste com menos parâmetros e, assim, tornar o processo de ajuste fino mais computacionalmente viável, eficiente e sustentável (HAN et al., 2024). É importante notar que, mesmo com um número expressivamente menor de parâmetros, os métodos de PEFT apresentam um desempenho comparável, mas tipicamente inferior, ao ajuste fino completo em tarefas de linguagem (DING et al., 2023).

Um dos métodos de PEFT mais adotados na literatura para modelos de arquitetura *transformer* é o *Low-Rank Adaptation* (LoRA) (HU et al., 2022). Apoiado na tese de que os modelos superparametrizados, em sua essência, residem em uma dimensão intrínseca baixa, o LoRA é uma técnica que propõe que as matrizes de pesos sejam ajustadas por uma matriz de *rank* baixo, oriunda de uma decomposição.

Dado o sucesso do LoRA, que, com pouquíssimos parâmetros, alcança performance

preditiva similar ao de um ajuste completo, inúmeras técnicas surgiram buscando aprimorá-lo. Apesar das diversas abordagens efetivas existentes, como alocação adaptativa de parâmetros (ZHANG et al., 2023), métodos baseados em *mixture-of-experts* (TIAN et al., 2024; FAN et al., 2025) e técnicas para aumentar o *rank* do ajuste (HUANG et al., 2025), a avaliação do impacto dos módulos adicionados a cada camada permanece pouco explorada na literatura.

Portanto, investigar a influência para a performance preditiva do modelo do uso de módulos LoRA em camadas específicas no processo de ajuste-fino mostra-se relevante não só pela importância do ajuste eficiente de parâmetros para o desenvolvimento sustentável de aprendizado de máquina, como também pela contribuição que a exploração de uma abordagem não usual pode trazer para a área.

1.2 Objetivos

Este projeto busca propor, implementar e analisar um método de PEFT baseado na técnica LoRA. Em contraste com as abordagens existentes, este trabalho explora como os módulos LoRA específicos de cada camada influenciam a habilidade preditiva do modelo, mantendo módulos apenas em posições que apresentem impacto significativo. Desta forma, da perspectiva prática, o projeto prevê uma minimização da adição de parâmetros de pouco impacto através da identificação de módulos que alteram de modo insignificante ou menor a representação gerada pelo modelo, o que se traduz em menor demanda por infraestrutura computacional, bem como em redução do consumo energético. Para o reconhecimento dos módulos que trazem mudanças significativas, utiliza-se de métricas de similaridade de representação de estruturas de redes, em especial, neste trabalho é dada ênfase para o *Centered Kernel Alignment* (CKA) (KORNBLITH et al., 2019). Da perspectiva teórica, o trabalho pretende responder a seguinte pergunta de pesquisa: *É possível, sistematicamente, escolher as camadas mais importantes para o ajuste fino em uma tarefa?* A resposta para essa pergunta, assim como o uso de métricas de similaridade para este fim, explora uma via pouco comum na literatura.

1.3 Justificativa

A literatura moderna sugere que as camadas de um grande modelo de linguagem não contribuem de forma homogênea para a representação final gerada e, consequentemente, para o seu desempenho em determinada tarefa. Além de abordar o problema com evidências experimentais para confirmar a hipótese de que as camadas não realizam contribuições iguais, como sugere a Figura 1, é possível investigar o comportamento das camadas a partir da análise de suas representações. Métricas de similaridade, como *Centered Kernel Alignment* (CKA) (KORNBLITH et al., 2019), ou similaridade por cosseno, viabilizam

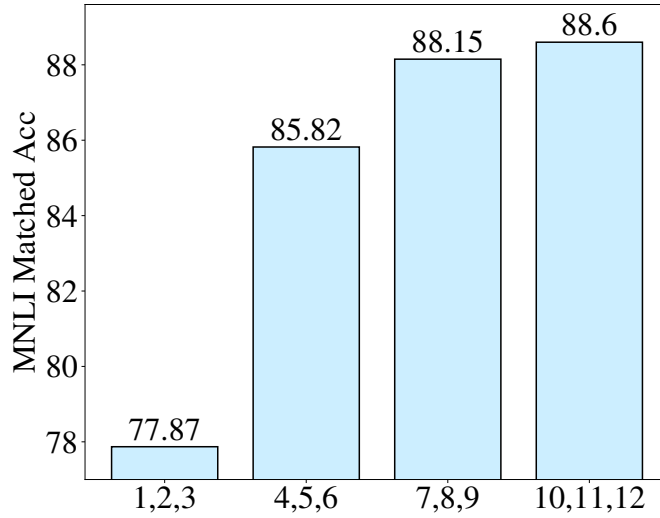


Figura 1 – Figura retirada do paper *AdaLoRA: Adaptive Budget Allocation For Parameter-Efficient Fine-Tuning* (ZHANG et al., 2023) ilustrando desempenho do DeBERTaV3-base no *benchmark* MNLI-m, sendo ajustadas apenas as camadas escolhidas.

o estudo e identificação de fenômenos internos do modelo. Por exemplo, a saturação de representação (GEVA et al., 2022), em que, em cenários específicos, a partir de determinada camada, o modelo deixa de adicionar informação de forma efetiva nas suas representações internas.

Dessa forma, justificamos o trabalho não só pela contribuição para o avanço científico, gerado pela exploração da perspectiva de similaridade para atribuir uma importância de uma camada para a performance de um modelo, mas também pela sua relevância para o desenvolvimento sustentável e acessível de aprendizado de máquina. Este aspecto é decorrente tanto da redução de parâmetros treinados quanto da sua ortogonalidade com métodos baseados no LoRA que focam em estruturas internas das camadas, o que possibilita aplicações simultâneas, possivelmente resultando em cenários de ainda mais eficiência.

1.4 Definição do Problema Abordado

A partir das motivações acima, o objetivo e as justificativas do trabalho, é possível definir o problema abordado:

Seja $\mathcal{F}$ um modelo de arquitetura *transformer* previamente treinado composto de $\mathcal{C}$ camadas nas quais serão inseridos módulos LoRA para o ajuste fino, deseja-se, através do uso de uma métrica m , identificar, em quais N camadas, os módulos LoRA trazem maior impacto na performance preditiva do modelo para uma tarefa específica.

2 Aspectos Conceituais

Este capítulo apresenta os principais conceitos utilizados no desenvolvimento do trabalho, abordando a arquitetura *transformers*, ajuste fino eficiente de grandes modelos de linguagem, em especial o método LoRA e uma de suas variantes, e métricas de similaridade, dando ênfase para o *Centered Kernel Alignment*. Por fim, incluímos as soluções existentes na literatura para seleção de camadas para o ajuste fino.

2.1 Arquitetura *transformer*

Arquitetura Transformer é um termo que engloba uma família de arquiteturas de redes neurais profundas baseadas na originalmente proposta na publicação *Attention is All You Need* (VASWANI et al., 2017). A arquitetura original segue um modelo baseado em dois grandes blocos: O *encoder* e o *decoder*. Seus sucessores utilizam, em alguns casos, apenas o *decoder*, em outros, apenas o *encoder*, ou ambos. Descreveremos inicialmente a arquitetura seminal, seus mecanismos são a base para compreender o funcionamento das demais.

2.1.1 Arquitetura *transformer* encoder-decoder

Em sua concepção inicial, a arquitetura *transformer* foi desenvolvida para resolver a tarefa de tradução de textos, sendo composta de dois blocos: o *encoder* e o *decoder*, cada um de camadas iguais repetidas. Assim, o *encoder* é responsável por extrair da entrada uma representação significativa para ser utilizada pelo bloco *decoder*. Já o *decoder* é responsável por receber a representação gerada pelo *encoder* e, iterativamente, gerar símbolos, de forma autoregressiva, ou seja, consumindo a saída da iteração anterior para produzir a iteração atual. A Figura 2 ilustra a arquitetura, o bloco *encoder* encontra-se à esquerda e o *decoder* à direita. A entrada de cada um deles inclui uma representação contínua construída a partir do texto discreto pela camada de *embedding* e pelo codificador posicional. A seguir, descrevemos a camada de *embedding*, o codificador posicional e os módulos que compõem ambos os tipos de blocos.

2.1.1.1 Camada de *embedding* e Tokenizador

A arquitetura *transformer* herdou características e mecanismos de outros modelos utilizados para processamento de linguagem natural. Assim como em modelos anteriores, a arquitetura não opera diretamente sobre palavras, mas sobre pedaços delas denominados

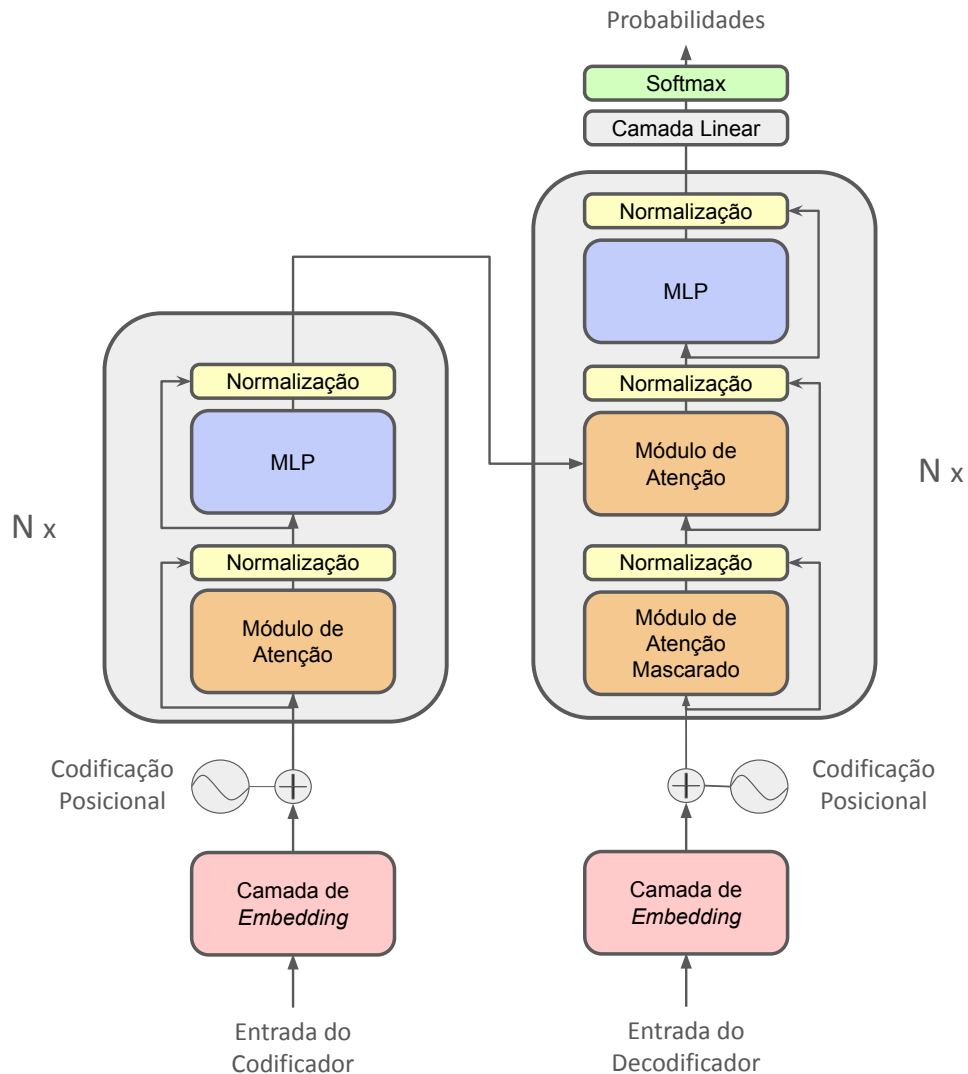


Figura 2 – Arquitetura Encoder-Decoder, ilustração inspirada no trabalho *Attention is All You Need* (VASWANI et al., 2017)

tokens. Os modelos possuem seu vocabulário de *tokens*, e o *tokenizador* transforma toda entrada em uma sequência deles.

Estes *tokens* são pedaços que não carregam de forma natural informação sobre si. Para um melhor processamento no modelo, a camada de *embedding* é responsável por mapear estas unidades discretas para um espaço dimensional maior (espaço dos *embeddings*), de forma que sua posição nele contém informação. Na arquitetura *transformer* original, e tipicamente nas demais, a camada de *embedding* é uma estrutura já treinada para gerar estes vetores com representação semântica do *token*.

2.1.1.2 Codificador Posicional

Além da informação semântica de cada *token*, é desejável que o modelo receba informação quanto à posição dele em relação aos demais, uma vez que a ordem dos *tokens* influencia o significado do texto de entrada.

Para isso, o modelo usa um codificador posicional para inserir informação de posição em cada *token*. Nesse caso, o codificador posicional é composto de funções seno e cosseno, possui mesma dimensão da saída da camada de *embedding* e soma-se a ela.

2.1.1.3 Módulos

As camadas no interior dos blocos *encoder* e *decoder* são compostas de módulos de atenção, módulos *Multilayer Perceptron* (MLP) e de conexões residuais e normalizações. Portanto, em uma perspectiva de menor granularidade, descreve-se a arquitetura como uma sequência de unidades compostas por essas estruturas. A seguinte equação mostra o formato de um destes componentes:

$$\text{LayerNorm}(x + \text{Módulo}(x)).$$

Com n_{token} sendo o número de *tokens* e d_{model} a dimensão interna do modelo, que é constante entre camadas na arquitetura transformer, $x \in \mathbf{R}^{n_{token} \times d_{model}}$ é a entrada do módulo e *Módulo()* um dos possíveis módulos que descrevemos nas próximas seções.

2.1.1.3.1 Módulo de atenção

O principal diferencial do *transformer* em relação às arquiteturas que antes eram estado da arte é a presença e foco em um módulo de atenção sem recorrência na arquitetura.

Três vetores descrevem seu mecanismo de atenção: *Key*, *Query* e *Value*. Estes são calculados para cada entrada do módulo, ao multiplicar as entradas com as matrizes associadas a cada um deles (W_Q, W_K, W_V) pela esquerda. Então, realiza-se o produto escalar entre as *Keys* e as *Queries*, normalizado pela raiz da dimensão desses vetores (d_k) e aplica-se uma função para obter os pesos dos *Values* que são somados para gerar o vetor de saída do módulo de atenção. A função aplicada é a *softmax*, a qual transforma um vetor de números reais g de dimensão d_g em uma distribuição de probabilidades da seguinte forma:

$$\text{softmax}(g_i) = \frac{e^{g_i}}{\sum_{j=1}^{d_g} e^{g_j}}. \quad (2.1)$$

Nota-se que calculamos a atenção de modo simultâneo, assim, os vetores são agrupados em matrizes Q, K, V e a atenção pode ser descrita como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

É possível perceber que a saída do módulo de atenção, nesse caso, é um vetor com elementos de dimensão da dimensão do *Value* (d_v), que, caso fosse um único módulo, deveria ser a mesma dimensão dos *embeddings* do modelo. Entretanto, executa-se a atenção em diversos módulos menores denominados de *Attention-Heads*. No artigo original, utilizam-se as seguintes dimensões para Q, K, V : $d_k = d_v = \frac{d_{model}}{8}$. Podendo, então, calcular a saída do módulo de atenção em termos da concatenação das saídas parciais:

$$\text{Atenção}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O,$$

com

$$\text{attention-head}_i = \text{Atenção}(QW_i^Q, KW_i^K, VW_i^V).$$

Sendo as matrizes de projeções para os sub-módulos de atenção: $W_i^Q \in \mathbf{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbf{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbf{R}^{d_{model} \times d_v}$.

A Figura 2 ilustra que, enquanto as camadas do *encoder* possuem apenas um módulo de atenção, as camadas do *decoder* possuem dois módulos, sendo um deles mascarado e um deles recebendo vetores do *encoder*. Primeiro, o módulo de atenção mascarado é um módulo de atenção com uma máscara no argumento da função *softmax*, permitindo apenas produtos escalares entre *Keys* e *Queries* até o token gerado na iteração atual. Isso para evitar que o modelo receba informação de tokens futuros durante o treinamento de um processo autoregressivo. Então, há um módulo de atenção sem máscara, mas que recebe *Keys* e *Values* do *Decoder* para incorporar esta informação previamente extraída.

2.1.1.3.2 Módulo *Multilayer Perceptron* (MLP)

O módulo *Multilayer Perceptron* (MLP) consiste em uma rede neural de mais de uma camada, a qual é composta por transformações lineares e funções não lineares, denominadas funções de ativação, intermediando-as. Neste caso, a rede possui duas camadas e a função da ativação é a *Rectified Linear Unit* (ReLU). Podemos descrever a saída deste módulo da seguinte forma:

$$\text{MLP}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2.$$

Em que $W_1 \in \mathbf{R}^{d_{model} \times d_{inter}}$, $W_2 \in \mathbf{R}^{d_{inter} \times d_{model}}$ são as matrizes das transformações lineares e $b_1 \in \mathbf{R}^{d_{inter}}$, $b_2 \in \mathbf{R}^{d_{model}}$ são os termos de vies. Com d_{model} sendo a dimensão interna do modelo e d_{inter} a dimensão intermediária, interna do módulo MLP.

2.1.2 Arquitetura *transformer encoder-only*

Após o pioneirismo do modelo original, inúmeras variações surgiram, uma das famílias é a dos *encoder-only*, com membros como os modelos RoBERTa (LIU et al.,

2019) e DeBERTa-v3 (HE et al., 2021). Estes modelos utilizam apenas o bloco de *encoder*, portanto, não são modelos autorregressivos. Normalmente, modelos *encoder-only* são utilizados em tarefas baseadas em compreensão de texto, dado que analisam a entrada como um todo, sem nenhum tipo de máscara no módulo de atenção.

2.1.3 Arquitetura *transformer decoder-only*

Além dos modelos *encoder-only*, também há aqueles que têm, em sua arquitetura, apenas o bloco *decoder* como Mistral (JIANG et al., 2023), LLaMA (TOUVRON et al., 2023) e Gemma (TEAM et al., 2024). Estes têm natureza autorregressiva e são utilizados para tarefas de geração de textos.

É importante notar que os modelos *encoder-only* não utilizam o bloco idêntico ao da arquitetura original, seus blocos possuem apenas o módulo de atenção com máscara, descartando o módulo de atenção que receberia as informações do bloco *decoder*.

2.1.4 Arquiteturas multimodais

Além dos grandes modelos de linguagem, existem também arquiteturas derivadas da *transformer* que operam não só no domínio de texto, mas também em outros, como visão e áudio. Por exemplo, o modelo LLaVA (LIU et al., 2023) apresenta uma arquitetura específica para operar com dados de visão e linguagem.

Em especial, o LLaVA destaca-se por seu pioneirismo em seguir instruções com informações visuais e em texto. O modelo possui em sua arquitetura partes disjuntas de visão e linguagem. O codificador de visão é responsável por extrair representações das imagens, as quais são adaptadas para o formato dos *embeddings* de texto pela camada de projeção. Já o modelo de linguagem é responsável por utilizar a instrução em texto e as representações extraídas da imagem para gerar uma resposta em texto. A Figura 3 ilustra a arquitetura.

2.2 *Parameter Efficient Fine-tuning (PEFT)*

Parameter Efficient Fine-tuning é um termo que engloba um conjunto de técnicas que envolvem realizar o ajuste fino de uma pequena porção dos parâmetros totais de um modelo, ou de uma baixa quantidade adicional de parâmetros, visando alcançar uma performance comparável com a do ajuste total (DING et al., 2023). Esta seção descreve os métodos LoRA e PiSSA.

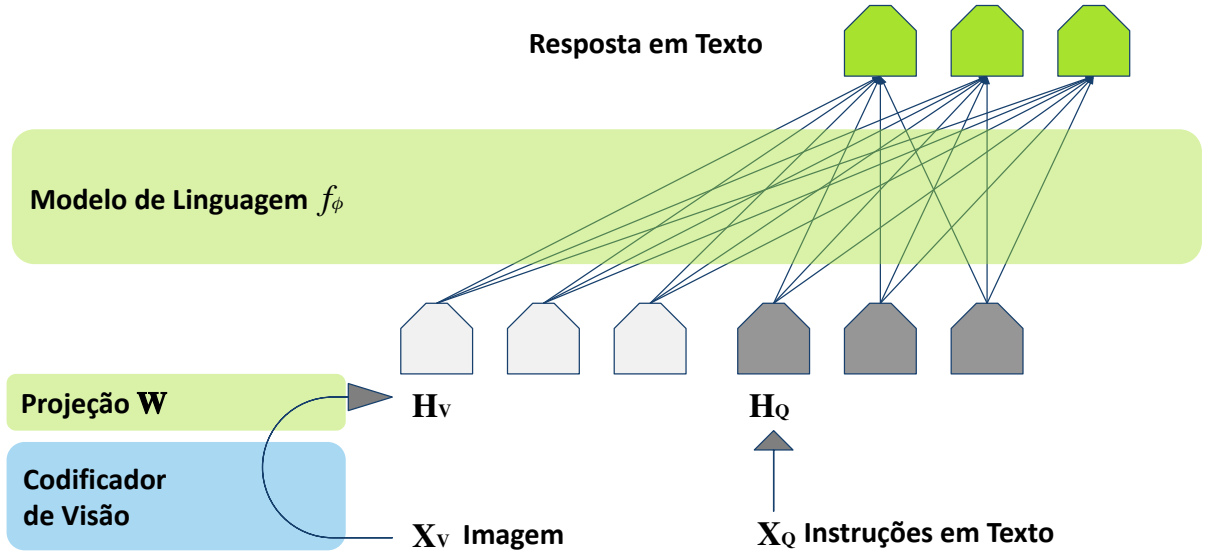


Figura 3 – Arquitetura do modelo LLaVA, ilustração inspirada no trabalho *Visual Instruction Tuning* (LIU et al., 2023)

2.2.1 Low-Rank Adaptation (LoRA)

O mecanismo LoRA (HU et al., 2022) é uma técnica de ajuste fino eficiente que, baseada na premissa de que os modelos treinados residem em uma dimensão intrínseca baixa, busca realizar ajustes também de dimensão intrínseca baixa. Para atingir esse objetivo, o método realiza o ajuste através de uma decomposição matricial de uma matriz de baixo *rank*. Assim, em vez de realizar o ajuste ΔW nos pesos pré-treinados $W \in \mathbf{R}^{d \times k}$ como a soma de uma matriz treinável de mesma dimensão de W , o mecanismo LoRA decompõe ΔW como uma multiplicação de matrizes $B \in \mathbf{R}^{d \times r}$ e $A \in \mathbf{R}^{r \times k}$. A é inicializado de forma aleatória com uma distribuição gaussiana e B inicializado com zeros para que, no início do treino, apenas o peso original esteja presente. Dessa forma, dado que $r \ll \min(d, k)$, a quantidade de parâmetros treináveis é bruscamente reduzida de dk para $r(d + k)$. Ao fim do processo de ajuste-fino, ΔW é somado a W_0 para gerar os pesos pós-ajuste W :

$$W = W_0 + \Delta W = W_0 + BA$$

Atualmente, o LoRA é uma das técnicas de ajuste fino eficiente mais populares, não só devido à pouca queda de desempenho preditivo mesmo com uma queda de parâmetros significativa, mas também devido à ausência de latência adicional de inferência, uma vez que as matrizes são somadas aos pesos originais ao fim do processo de ajuste fino.

2.2.2 Principal Singular Values and Singular Vectors Adaptation (PiSSA)

Construindo a partir do LoRA, o PiSSA (MENG; WANG; ZHANG, 2024) explora a inicialização das matrizes treináveis A e B que compõem a atualização ΔW . Em contraste com a inicialização aleatória em A e com zeros em B , o método busca utilizar informações presentes na matriz de peso W através da identificação dos maiores valores singulares da decomposição em valores singulares (SVD, do inglês *Singular Value Decomposition*). Ao utilizar o PiSSA, realiza-se o SVD da matriz W e inicializam-se as matrizes A e B com os vetores com valores singulares de maior magnitude. A parte congelada da matriz torna-se o resíduo com os demais vetores e valores singulares menores. Os experimentos do trabalho de Meng et al. 2024 mostram que o PiSSA apresenta não só uma convergência mais rápida, mas também melhor desempenho preditivo nas tarefas abordadas. Por fim, é importante notar que, dado que o PiSSA só altera a inicialização do LoRA, características do método, como atuação a nível de matriz, se mantêm.

2.3 Métricas de similaridade de representação

Durante o processo de inferência, as camadas das redes neurais geram sucessivamente representações extraídas dos dados de entrada, até a geração da representação final, que é dada como saída da rede ou utilizada para gerá-la. Métricas de similaridade são métodos que geram um valor que expressa o grau de similaridade entre duas representações. Esta seção descreve uma das métricas de similaridade mais populares da literatura de redes neurais: *Centered Kernel Alignment* (CKA).

2.3.1 Centered Kernel Alignment (CKA)

Proposto para atender ideais de uma métrica de similaridade não obtidos em esforços anteriores, o *Centered Kernel Alignment* (KORNBLITH et al., 2019) é uma métrica baseada em, primeiro, medir a similaridade entre todos os pares de exemplos em cada uma das representações e, então, comparar as estruturas de similaridade, diferente de outros métodos que realizam a comparação diretamente entre os exemplos das representações individuais.

Tal método é possível através de uma construção baseada no critério de independência de Hilbert-Schmidt (HSIC, do inglês *Hilbert-Schmidt Independence Criterion*) (GRETTON et al., 2005). Este critério foi desenvolvido como um teste estatístico para definir a independência entre duas variáveis com o uso de *kernels*, sendo uma abordagem bem fundamentada na literatura utilizá-lo para entender a similaridade ou dependência entre estruturas. Tomando o HSIC implementado com *kernels* $\mathbf{k}$ e $\mathbf{l}$ definidos, e $\mathbf{X}$, $\mathbf{Y}$ como as matrizes de representação centralizadas, é possível descrever o CKA com $K_{ij} = k(x_i, x_j)$ e $L_{ij} = l(y_i, y_j)$ como:

$$CKA(K, L) = \frac{HSIC(K, L)}{\sqrt{HSIC(K, K)HSIC(L, L)}}.$$

Nota-se que o CKA envolve também a normalização do critério, isso pois, dessa forma, é possível atingir características desejadas em métricas de similaridade como invariância às transformações ortogonais e transformações isotrópicas de escala.

2.4 Seleção de Camadas para Ajuste Fino

Existem trabalhos anteriores a este que abordam a seleção de camadas o ajuste fino. No domínio de visão, Lee et al. 2023 cunharam o termo *Surgical Fine-tuning*, referindo-se ao ajuste fino completo de subconjuntos de sequências contíguas de camadas de modelos convolucionais. Na próxima seção discutiremos a extensão deste conceito para os modelos de linguagem, bem como as diferenças com a abordagem do nosso trabalho.

2.4.1 Seleção de Camadas para modelos *encoder-only*

Estendendo o conceito de *surgical fine-tuning* para o domínio de linguagem, Lodha et al. 2023 propuseram o uso do *Fisher Information Matrix* (FIM) para identificar as camadas para o treino. O FIM é uma matriz que contém informação de como a alteração em um parâmetro do modelo influencia na previsão dele. Os autores constroem seu índice de importância da camada como a agregação dos valores da matriz referentes à camada.

Apesar de carregar o nome *surgical fine-tuning*, o trabalho com o uso do FIM não aborda sequências contíguas de camadas, mas sim a seleção de um subconjunto específico de camadas *transformer*. Dessa forma, aborda o mesmo problema que tratamos neste trabalho. Entretanto, as pesquisas diferem em abordagem, modelos e tarefas compreendidas. Especificamente, enquanto o método com FIM trata do ajuste completo de camadas selecionadas com passos de treino, focamos na seleção de camadas com inferências como um complemento eficiente a métodos como LoRA. Ademais, Lodha et al 2023 limitam-se a modelos *encoder-only* e tarefas de compreensão de texto, já neste trabalho, avaliamos nosso método com modelos *encoder-only*, *decoder-only* e multimodais em não só tarefas de compreensão de texto, mas também tarefas complexas de geração de texto e perguntas e respostas. Além da diferença em escopo, contrastamos nosso método com o uso do FIM nos experimentos da Seção 4.2.

3 Método Proposto

3.1 Método proposto

Construindo em cima da hipótese de que o conjunto ideal de camadas para o ajuste-fino de um modelo é específico ao dado no qual ele será treinado, utilizamos as representações dos modelos para identificar as camadas seguindo o fluxo abaixo.

Primeiro, extraímos as representações internas da saída de cada camada e da saída da camada de *embedding*. Em consonância com Klabunde et al. 2025, argumentamos que, entre as opções de *tokens* para usar a representação, em modelos *encoder-only* o *token* $\langle \text{CLS} \rangle$ é o mais representativo e em modelos *decoder-only* o último token, o qual é o gerado na inferência, é o mais representativo. Estes *tokens* são diretamente relacionados com as tarefas realizadas pelos modelos, os modelos *encoder-only* utilizam o *token* $\langle \text{CLS} \rangle$ para alimentar a cabeça de classificação e os modelos *decoder-only* utilizam o último *token* como o gerado na inferência, sendo concatenado na entrada para o processo autoregressivo de geração de texto.

Em seguida, utilizamos as representações para atribuir uma medida de importância pra cada camada. Sendo C_i a camada i do modelo e R_i a representação interna de sua saída (tomamos a saída da camada de *embedding* como R_0), nossa métrica de importância mapeia C_i para um valor real $I_{C_i} \in [0, 1]$, que descrevemos como a dissimilaridade entre as representações de entrada e saída de C_i :

$$I_{C_i} = 1 - \text{CKA}(R_{i-1}, R_i).$$

Esta formulação visa quantificar o impacto que cada camada tem nas mudanças das representações internas através da métrica de similaridade CKA. Toma-se que, dado uma camada i , caso a sua representação de entrada R_{i-1} seja similar a sua representação de saída R_i , este bloco contribuiu pouco para a construção da representação final utilizada na tarefa do modelo. Então, com valores de importância atribuídos para toda camada do modelo, selecionamos as N camadas mais importantes para receberem o ajuste fino. É importante notar que o método não assume nada quanto a técnica de ajuste, portanto, pode ser facilmente integrado em qualquer processo de ajuste fino. A Figura 4 ilustra o método, o qual se apoia na medida isolada da contribuição de cada camada.

Finalmente, cabe destacar que este processo de seleção das camadas para o ajuste-fino não é computacionalmente custoso. Em contraste com outros métodos que realizam a adaptação durante o treino ou na execução de passos de treino para atribuir importância

para parâmetros, nossa abordagem apresenta uma seleção por meio de inferências e cálculo do CKA pré-treino, apresentando um baixo custo computacional.

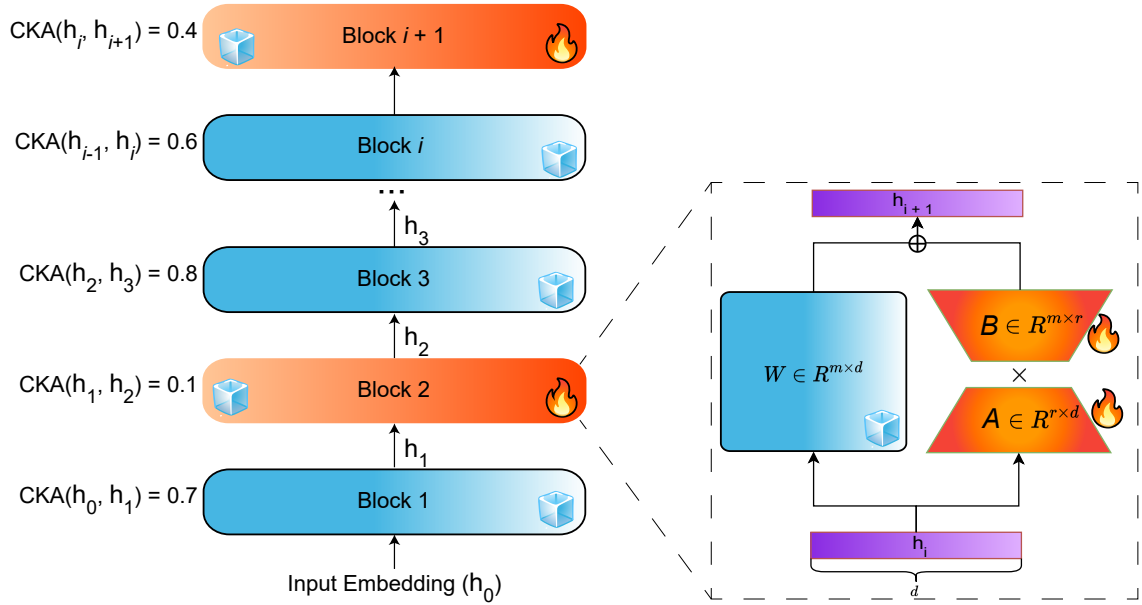


Figura 4 – Visão geral do nosso método. Ajustamos apenas as camadas com menor valor de CKA entre representações de entrada e saída (partes em vermelho). Mantemos o termo *Block* para preservar a consistência com a terminologia da literatura.

4 Experimentos

Avaliamos nosso método em um conjunto diverso de configurações. Tratamos de tarefas de compreensão e geração de texto, de modelos de diferentes naturezas e tamanhos e de aspectos específicos do nosso método. A seguir, descrevemos não só estes experimentos, como também as tecnologias utilizadas para implementá-los.

4.1 Tecnologias utilizadas

Durante o desenvolvimento e experimentação da técnica de seleção de camadas para ajuste-fino, utilizamos diversas tecnologias e ferramentas para propor, implementar e estudar uma solução para o problema abordado. A seguir, apresentamos aquelas consideradas essenciais para a condução da pesquisa.

4.1.1 Python

Python é uma linguagem de programação interpretada de alto nível de abstração. Com sua sintaxe simples e uma grande comunidade que a mantém, é uma linguagem popular no cenário de desenvolvimento de modelos de aprendizado de máquina. Esta popularidade aliada à existência de bibliotecas maduras para operações com tensores e desenvolvimento de modelos de linguagem justifica a escolha dela para nosso trabalho.

4.1.2 PyTorch e bibliotecas do *HuggingFace*

Em nosso trabalho, optamos pelo uso do popular *framework* para operações com tensores em Python: PyTorch. Além disso, utilizamos o ecossistema desenvolvido pela empresa *HuggingFace*, em especial, as bibliotecas *transformers*, *peft* e *datasets*. A biblioteca *transformers* disponibiliza não só modelos baseados na arquitetura *transformers*, mas também as ferramentas necessárias para empregá-los, como otimizadores e *tokenizers*. Complementando-a, a biblioteca *peft* disponibiliza implementações de métodos de ajuste-fino, como o LoRA, para o uso no treino destes modelos. Por fim, a biblioteca *datasets* possibilita o acesso a bases de dados essenciais para o treino e avaliação dos modelos.

4.2 Avaliação em compreensão de texto

Em compreensão de texto, ajustamos os modelos RoBERTa_{base} (LIU et al., 2019) e DeBERTa-v3_{base} (HE et al., 2021) no *benchmark* GLUE (WANG et al., 2019), seguindo a configuração experimental do LoRA (HU et al., 2022) e do PiSSA (MENG; WANG;

ZHANG, 2024) para uma comparação justa. Na execução do nosso método e do proposto por Lodha et al. 2023, selecionamos metade das camadas de cada modelo para serem ajustadas. A Tabela 3 mostra os resultados dos experimentos.

Observamos que o ajuste-fino apenas nos blocos selecionados pelo nosso método apresenta resultados com uma queda de desempenho preditivo marginal, mesmo com uma redução de 50% dos parâmetros treináveis do modelo. Em particular, com o modelo DeBERTa-v3_{base}, alcançamos essa redução com uma queda em média de apenas 0.27 pontos percentuais em relação ao LoRA convencional. Em contraste, o método existente na literatura, o qual se baseia no cálculo do FIM sobre passos de treino (LODHA et al., 2023), apresenta uma queda de 8.37 pontos percentuais.

Por sua vez, com o modelo RoBERTa_{base}, apesar do nosso método também superar o existente na literatura, a diferença média em desempenho preditiva é pequena. A Figura 5 ilustra as diferenças por tarefa e em média. Acreditamos que esse resultado ocorre devido ao modelo apresentar grande parte dos seus parâmetros fora das camadas, o que possivelmente diminui o impacto da seleção de camadas na performance preditiva do modelo ajustado. Independentemente desta menor diferença, de modo geral, o método apresenta resultados que confirmam a efetividade da seleção de camadas para modelos *encoder-only* em tarefas de compreensão de texto.

Além disso, é visível que existem tarefas em que a diferença de desempenho preditivo entre o ajuste em todas as camadas e em camadas selecionadas é mais acentuada. Observamos novamente e discutimos este fenômeno na Seção 4.5.

Finalmente, é importante destacar que, para os modelos *encoder-only*, não selecionamos a primeira camada em nosso método. Foi tomada essa decisão por observarmos que o método não quantifica de forma adequada a importância dela. O Apêndice A detalha

Modelo & Método	Parâmetros	MNLI	SST2	MRPC	CoLA	QNLI	QQP	RTE	STSB	Média
DeBERTa-v3 _{base} (Ajuste-fino completo)	184M	89.90	95.63	89.46	69.19	94.03	92.40	83.75	91.60	88.25
DeBERTa-v3 _{base} (LoRA)	1.33M	90.65	94.95	89.95	69.82	93.87	91.99	85.20	91.60	88.50
DeBERTa-v3 _{base} (LoRA + FIM)	0.66M	86.85	91.44	78.43	61.11	92.32	90.69	58.24	85.46	80.82
DeBERTa-v3 _{base} (LoRA + Nosso)	0.66M	89.93	94.84	89.38	67.16	93.80	91.93	87.84	90.90	88.23
RoBERTa _{base} (Ajuste-fino completo)	125M	87.60	94.80	90.20	63.60	92.80	91.90	78.70	91.20	86.40
RoBERTa _{base} (LoRA)	0.3M	87.50	95.10	89.70	63.40	93.30	90.80	86.60	91.50	87.2
RoBERTa _{base} (LoRA + FIM)	0.15M	86.90	93.65	89.22	58.73	93.01	89.67	71.60	90.45	84.16
RoBERTa _{base} (LoRA + Nosso)	0.15M	86.70	93.60	88.60	61.30	92.80	89.70	74.50	90.20	84.70

Tabela 1 – Resultados dos modelos RoBERTa_{base} e DeBERTa-v3_{base} ajustados no GLUE *benchmark*. Utilizamos como métricas de avaliação: a correlação de Matthew para o CoLA, a correlação de Pearson para o STSB e a acurácia para as demais tarefas. Resultados do modelo DeBERTa-v3_{base} com ajuste-fino completo e LoRA (HU et al., 2022) são oriundos do trabalho do método PiSSA (MENG; WANG; ZHANG, 2024), e correspondem à média de três execuções. Obtivemos os resultados do modelo RoBERTa_{base} com ajuste-fino completo e com LoRA do trabalho do LoRA (HU et al., 2022) como a mediana de cinco execuções. Produzimos os demais resultados como a média de três execuções.

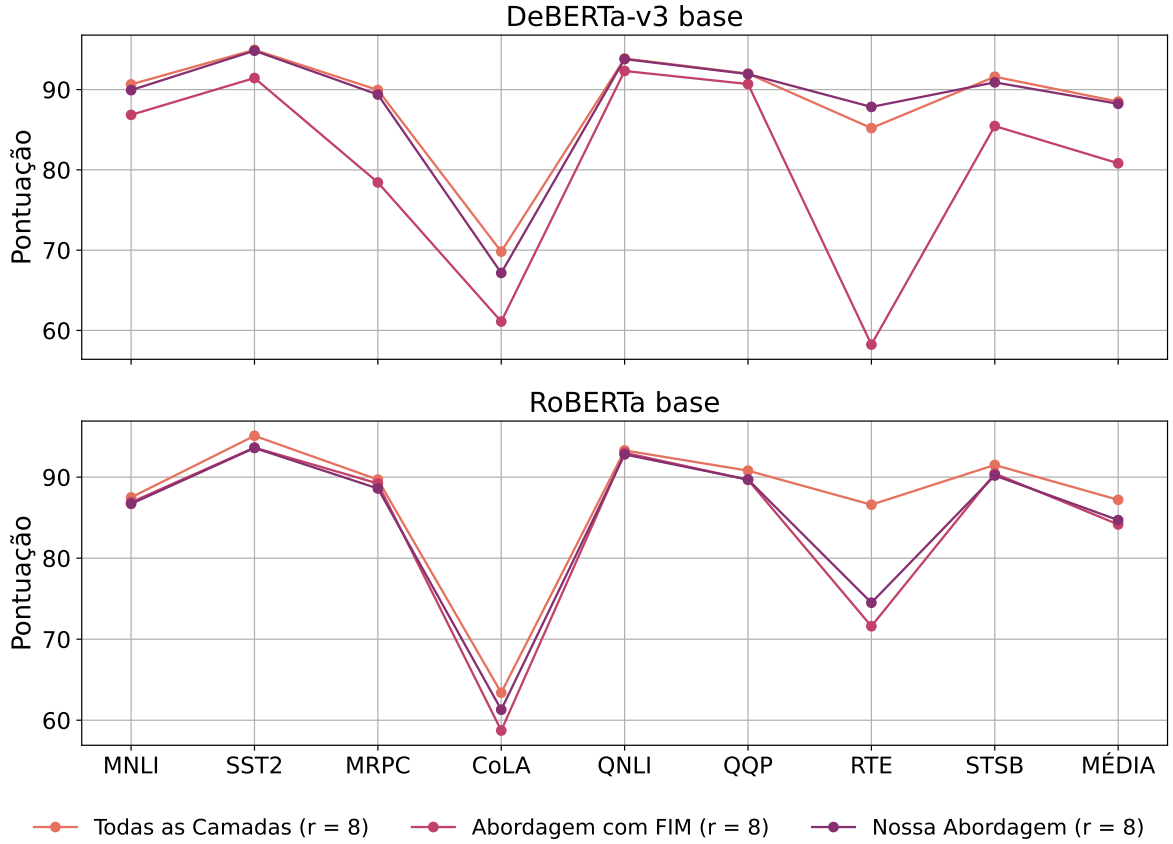


Figura 5 – Compilado de resultados dos modelos $\text{RoBERTa}_{\text{base}}$ e $\text{DeBERTa-v3}_{\text{base}}$ ajustados no GLUE *benchmark*.

essa questão.

4.3 Avaliação em geração de texto

Para avaliarmos nosso método em tarefas de geração de texto, ajustamos os modelos *decoder-only* LLaMA 2-7B (TOUVRON et al., 2023), Mistral-7B-v0.1 (JIANG et al., 2023), e Gemma-7B (TEAM et al., 2024) para tarefas de matemática e de geração de código. Para avaliação com matemática, ajustamos os modelos nos dados da base MetaMathQA (YU et al., 2024) e avaliamos nas partições de validação das bases GSM8k (COBBE et al., 2021) e MATH (HENDRYCKS et al., 2021). Para avaliação com geração de código, ajustamos os modelos nos dados da base Code-Feedback (ZHENG et al., 2024) e avaliamos nas bases HumanEval (CHEN, 2021) e MBPP (AUSTIN et al., 2021). Além disso, neste experimento, abordamos o uso do nosso método com outra técnica de ajuste fino eficiente além do LoRA: o PiSSA. A Tabela 2 mostra os resultados dos experimentos.

Em contraste com os resultados em compreensão de texto, os resultados em geração

Modelo	Parâmetros	Estratégia	GSM8K	MATH	HumanEval	MBPP
LLaMA 2-7B	6738M	Ajuste-fino completo	49.13±0.21	7.29±0.22	21.20±0.30	35.59±0.25
	320M	LoRA	42.85±0.12	5.50±0.33	18.35±0.31	35.50±0.14
	320M	PiSSA	53.22±0.55	7.47±0.34	21.92±0.38	37.24±0.63
	159M	LoRA + Nosso	47.28±0.44	6.45±0.06	21.77±0.97	37.67±0.40
	159M	PiSSA + Nosso	54.79±0.92	8.83±0.13	25.63±2.80	40.50±1.14
Mistral-7B	7240M	Ajuste-fino completo	69.91±0.25	18.64±0.35	45.31±0.14	51.46±0.13
	335M	LoRA	69.50±0.42	20.08±0.20	43.78±0.34	58.46±0.37
	335M	PiSSA	73.31±0.23	23.12±0.52	46.88±0.25	62.55±0.58
	167M	LoRA + Nosso	72.76±0.88	20.85±0.75	48.20±1.20	61.80±0.66
	167M	PiSSA + Nosso	72.66±0.46	20.77±0.28	49.40±0.60	60.90±0.17
Gemma-7B	8540M	Ajuste-fino completo	72.09±0.32	22.71±0.34	47.02±0.27	55.67±0.50
	400M	LoRA	75.11±0.64	30.41±0.48	53.70±0.23	65.58±0.29
	400M	PiSSA	77.78±0.32	31.33±0.33	54.31±0.28	66.17±0.43
	200M	LoRA + Nosso	74.58±0.38	28.59±0.38	51.56±2.33	62.07±1.94
	200M	PiSSA + Nosso	74.83±1.07	27.75±0.32	52.00±0.35	62.97±1.52

Tabela 2 – Resultados do LLaMA 2-7B, Mistral-7B-v0.1 and Gemma-7B. Utilizamos como métrica *pass@1* nos testes base para HumanEval e MBPP. Para o GSM8k e MATH utilizamos acurácia. Resultados de ajuste-fino completo, LoRA (HU et al., 2022) e PiSSA são do trabalho do PiSSA (MENG; WANG; ZHANG, 2024). Todos os resultados são a média de três execuções e apresentados com desvio padrão.

de texto não compartilham um mesmo padrão. Para os modelos LLaMA 2-7B e Mistral-7B, aplicar nosso método leva a uma redução de metade dos parâmetros treináveis com perda negligenciável ou até ganhos de performance preditiva. Em especial, observamos que na avaliação do LLaMA 2-7B no GSM8k o nosso método gera um ganho de 4.43 pontos percentuais de performance preditiva mesmo com redução de custos computacionais. Por outro lado, ao aplicarmos nosso método no Gemma-7B, observamos uma queda pequena na performance preditiva na maioria das tarefas. Este resultado traz indícios de que a seleção de camadas para o ajuste-fino é específica à arquitetura. Ademais, cabe ressaltar que a discussão acima é referente tanto à aplicação do nosso método com o LoRA quanto com o PiSSA. O comportamento homogêneo, para um mesmo modelo, com técnicas de ajuste fino diferentes reforça a argumentação quanto à característica agnóstica ao tipo de ajuste da seleção de camadas proposta. Assim, em suma, obtivemos resultados positivos para geração de texto com quedas mínimas ou até melhoras em desempenho preditivo.

4.4 Comparação com heurísticas simples

Para compreender a efetividade de I_{C_i} como métrica para seleção de camadas, substituímos nosso método por heurísticas simples. Acreditamos que um bom método deve equivaler ou superar estas estratégias, dado que elas poderiam ser utilizadas sem o custo, mesmo que pequeno, de um processo de escolha mais complexo. Realizamos os primeiros experimentos com modelos *encoder-only* no *benchmark* GLUE (WANG et al., 2019). Escolhemos as seguintes configurações de conjuntos de camadas (heurísticas):

camadas dos extremos, camadas centrais, primeiras camadas, últimas camadas e camadas alternadas. A Figura 6 ilustra estas configurações.

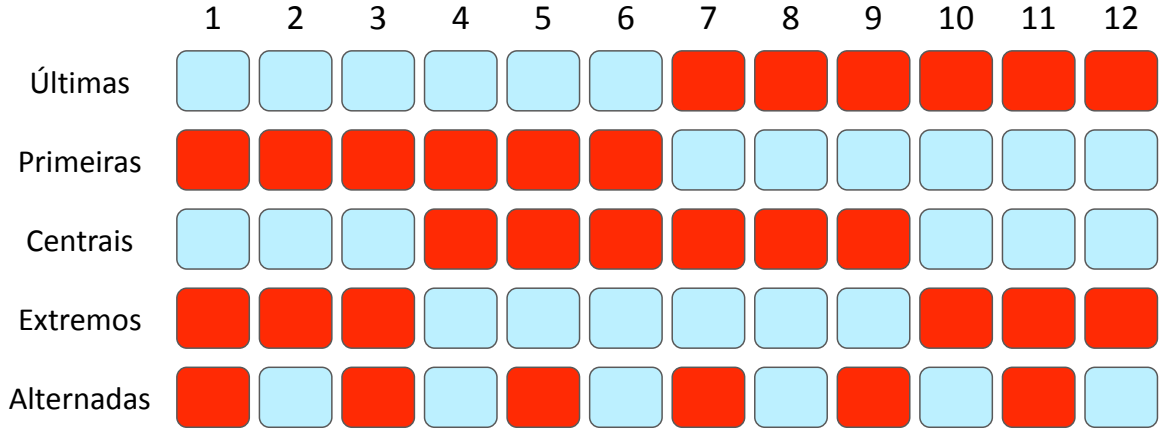


Figura 6 – Heurísticas de seleção de camadas para o ajuste-fino utilizadas como referência para os experimentos preliminares. Camadas vermelhas participam do treino e camadas azuis permanecem congeladas.

A Tabela 3 mostra que diferentes configurações de camadas ajustadas levam a modelos com diferentes performances preditivas. Assim, com uma diferença média de mais de 2 pontos percentuais entre a melhor e a pior configuração, os resultados sugerem que existem configurações de camadas selecionadas mais efetivas que outras, como suposto anteriormente.

Modelo & Método	Params	MNLI	SST2	MRPC	CoLA	QNLI	QQP	RTE	STSB	Média
RoBERTa _{base} (Full FT) ₁₁₁₁₁₁₁₁₁₁₁₁	125M	87.60	94.80	90.20	63.60	92.80	91.90	78.70	91.20	86.40
RoBERTa _{base} (LoRA) ₁₁₁₁₁₁₁₁₁₁₁₁	0.3M	87.50	95.10	89.70	63.40	93.30	90.80	86.60	91.50	87.2
RoBERTa _{base} (LoRA) ₀₀₀₀₀₀₁₁₁₁₁₁	0.15M	85.60	93.60	87.20	58.40	92.30	89.30	71.60	89.30	83.40
RoBERTa _{base} (LoRA) ₁₁₁₁₁₁₀₀₀₀₀₀	0.15M	85.50	90.80	87.20	55.00	91.10	89.30	69.30	89.20	82.20
RoBERTa _{base} (LoRA) ₀₀₀₁₁₁₁₁₁₀₀₀	0.15M	86.60	93.40	87.20	59.90	92.60	89.70	75.60	90.70	84.50
RoBERTa _{base} (LoRA) ₁₁₁₁₀₀₀₀₀₀₁₁₁	0.15M	86.00	94.00	86.00	58.00	92.80	89.40	69.90	89.50	83.20
RoBERTa _{base} (LoRA) ₁₀₁₀₁₀₁₀₁₀₁₀	0.15M	86.70	93.70	86.90	60.40	92.70	89.80	73.40	90.70	84.30
RoBERTa _{base} (LoRA + Nosso)	0.15M	86.70	93.60	88.60	61.30	92.80	89.70	74.50	90.20	84.70

Tabela 3 – Resultados do modelo RoBERTa_{base} com ajuste-fino no *GLUE benchmark* com diferentes configurações de camadas selecionadas. São utilizadas como métricas de avaliação: a correlação de Matthew para o CoLA, a correlação de Pearson para o STSB e a acurácia para as demais tarefas. Reportamos os resultados do ajuste-fino completo e do LoRA em todas as camadas a partir do trabalho do LoRA (HU et al., 2022), como a média de cinco execuções. Os demais resultados são de nossa autoria e correspondem à média de três execuções. Utilizamos como notação para indicar as camadas ajustadas do modelo um vetor *one-hot-encoded*, posições com 1 receberam ajuste e posições com 0 não receberam.

Além disso, nota-se que a configuração de maior performance preditiva varia entre tarefas. Com base nessa evidência experimental e em concordância com Jin et al. 2025,

Modelo	Parâmetros	Estratégia	GSM8K	MATH	HumanEval	MBPP
LLaMA 2-7B	159M	LoRA + Nosso	47.28±0.44	6.45±0.06	21.77±0.97	37.67±0.40
	159M	LoRA + Centrais	45.33±0.87	6.62±0.21	23.80±1.59	35.80±0.66
	159M	PiSSA + Nosso	54.79±0.92	8.83±0.13	25.63±2.80	40.50±1.14
	159M	PiSSA + Centrais	52.86±0.50	8.44±0.08	23.20±1.04	39.33±0.75
Mistral-7B	167M	LoRA + Nosso	72.76±0.88	20.85±0.75	48.20±1.20	61.80±0.66
	167M	LoRA + Centrais	71.95±0.53	20.69±0.55	40.63±2.89	62.23±0.29
	167M	PiSSA + Nosso	72.66±0.46	20.77±0.28	49.40±0.60	60.90±0.17
	167M	PiSSA + Centrais	72.80±0.57	21.51±0.08	46.53±2.48	61.30±0.82
Gemma-7B	200M	LoRA + Nosso	74.58±0.38	28.59±0.38	51.56±2.33	62.07±1.94
	200M	LoRA + Centrais	76.72±0.53	29.03±0.22	50.20±1.25	64.80±0.50
	200M	PiSSA + Nosso	74.83±1.07	27.75±0.32	52.00±0.35	62.97±1.52
	200M	PiSSA + Centrais	76.52±0.59	27.49±0.19	52.63±1.44	62.60±1.91

Tabela 4 – Resultados do LLaMA 2-7B, Mistral-7B-v0.1 and Gemma-7B usando nosso método e a seleção das camadas centrais. Utilizamos como métrica *pass@1* nos testes base para HumanEval e MBPP. Para o GSM8k e MATH utilizamos acurácia. Resultados em destaque indicam o melhor resultado para LoRA e PiSSA em cada modelo. Produzimos todos os resultados como a média de três execuções e apresentamos com desvio padrão.

que mostram que conceitos de diferentes complexidades são processados em camadas de profundidades distintas, reforçamos a hipótese de que a escolha de camadas é intimamente relacionada ao dado com o qual deseja-se treinar o modelo.

Por fim, é perceptível que a configuração com as camadas selecionadas no centro apresenta um desempenho superior às demais heurísticas simples. Este resultado é esperado, visto que, como destacado por Skean et al. 2025, as camadas intermediárias, atingidas por essa simples estratégia de seleção, apresentam um papel essencial na construção das representações dos modelos *transformers*. Apesar deste simples método de escolhas de camadas apresentar um desempenho consistentemente superior aos demais neste experimento, nota-se que nosso método apresenta um desempenho preditivo médio superior no *benchmark* GLUE.

Para melhor compreensão do contraste entre nosso método e a estratégia de seleção das camadas centrais, estendemos a comparação para os modelos *decoder-only*. A Tabela 4 mostra que, apesar da seleção das camadas centrais apresentar melhor desempenho preditivo com o modelo Gemma-7B, o uso do CKA exhibe resultados mais consistentes. Especificamente, enquanto nosso método limita-se a uma diferença de 2.73 pontos percentuais nos casos em que perde, a seleção das camadas centrais chega a uma diferença drástica 7.57 pontos percentuais em relação ao desempenho da nossa técnica com o modelo Mistral-7B no *benchmark* de programação *HumanEval*. Acreditamos que nosso método evita estas situações catastróficas ao identificar blocos que apresentam uma importância geral à tarefa abordada. Em contraste, a seleção fixa de certas camadas pode não incluir estas estruturas cruciais.

4.5 Quanto ao número de blocos selecionados

Em nossos experimentos anteriores, selecionamos metade das camadas para realizar o ajuste fino. Como observamos uma redução negligenciável de desempenho preditivo em relação ao ajuste em todas as camadas, naturalmente surge a pergunta: *Como nosso método se comporta para outros números de camadas selecionadas?*

Para responder a essa pergunta, avaliamos o modelo RoBERTa_{base} ajustado com diferentes quantidades de camadas selecionadas no *benchmark* GLUE. A Tabela 5 introduz os resultados e mostra um padrão interessante: tarefas diferentes apresentam números de camadas selecionadas ideais distintos. Por exemplo, enquanto a tarefa SST2 apresenta desempenho preditivo próximo ao maior entre as diferentes configurações apenas com uma camada selecionada, a tarefa RTE apresenta um desempenho preditivo diretamente proporcional ao número de camadas selecionadas. Em consonância com Jin et al. 2025, supomos que tarefas complexas, envolvendo diferentes conceitos, podem necessitar de mais camadas ajustadas para o modelo alcançar desempenho preditivo satisfatório por seu conhecimento estar distribuído nas camadas conforme a complexidade. Apesar disso, a maioria das tarefas alcança seu valor máximo ou próximo dele com três ou seis camadas selecionadas. Este resultado indica que apenas poucas camadas podem ser suficientes para compreender a adaptação para uma tarefa. Ademais, de forma independente do padrão citado, nossa técnica alcança uma queda pequena de desempenho preditivo nas diversas configurações de tarefas. Portanto, confirmamos que nosso método adequa-se a diferentes orçamentos computacionais, o que possibilita o uso de ajuste fino sob influência de diferentes restrições.

Modelo & Método	Parâmetros	Número de Camadas	MNLI	SST2	MRPC	CoLA	QNLI	QQP	RTE	STSB	Média
RoBERTa _{base} (LoRA + Ours)	0.025M	1	80.49	92.85	81.54	51.55	89.18	86.70	61.61	85.77	78.71
RoBERTa _{base} (LoRA + Ours)	0.075M	3	85.18	93.80	87.17	59.93	91.98	88.18	73.40	90.02	83.70
RoBERTa _{base} (LoRA + Ours)	0.15M	6	86.74	93.60	88.60	61.30	92.80	89.70	74.50	90.17	84.66
RoBERTa _{base} (LoRA + Ours)	0.22M	9	87.03	90.34	86.93	60.82	92.80	89.54	77.61	90.52	84.96

Tabela 5 – Resultados da seleção de diferentes quantidades de camadas utilizando nosso método com RoBERTa_{base} no *benchmark* GLUE. São utilizadas como métricas de avaliação: a correlação de Matthew para o CoLA, a correlação de Pearson para o STSB e a acurácia para as demais tarefas. Produzimos todos os resultados como a média de três execuções.

4.6 Quanto ao papel do *rank*

O hiperparâmetro r do LoRA não só define o *rank* máximo do ajuste, mas também o número de parâmetros dos módulos. Huang et al. 2025 associam o aumento do *rank* com aumento em expressividade do ajuste e, conseqüentemente, com ganhos em desempenho preditivo. Assim, conforme reduzimos a quantidade de parâmetros treináveis, a seguinte pergunta surge: *Redistribuir os parâmetros para as camadas selecionadas, por meio de um*

aumento no rank, em vez de retirá-los, traz ganhos de desempenho preditivo em comparação com o ajuste em todas as camadas?

Para responder a essa pergunta, utilizamos nosso método para selecionar camadas do modelo RoBERTa_{base} para aplicar módulos LoRA com o dobro do *rank* da configuração original. Dessa forma, esta configuração mantém o mesmo número de parâmetros da original, uma vez que a seleção de camadas e o aumento do *rank* se compensam em termos de parâmetros. Em nossos experimentos, nos certificamos de que a relação entre os hiperparâmetros *rank* e *alpha* do LoRA permaneça a mesma. A Tabela 6 apresenta os resultados e mostra que a redistribuição de parâmetros não se converte em um aumento efetivo de desempenho preditivo. Especificamente, as tarefas MRPC, CoLA, QNLI e STSB demonstram uma leve queda, enquanto as demais apresentam poucos ganhos em relação à seleção sem aumento em *rank*. Em geral, essas mudanças traduzem-se em um ganho médio de 0.09 pontos percentuais em comparação com a configuração original de *rank* oito.

Da nossa perspectiva, os resultados foram inesperados, uma vez que trabalhos recentes da literatura destacam a importância do *rank*. Entretanto, a configuração única com aumento de *rank* em apenas algumas camadas pode explicar essa diferença. Acreditamos que a razão por trás disso é a natureza da nossa abordagem. Como utilizamos a dissimilaridade de representações com múltiplas amostras, selecionamos camadas que interagem com características gerais dos dados. Nossa hipótese é que ajustes de baixo *rank* podem ser suficientes para abranger estes padrões gerais de maior granularidade, que tendem a ser menos complexos. Portanto, argumentamos que o aumento de *rank* torna-se ineficaz, pois, provavelmente, o método visa atingir interações que não necessitam da expressividade adicional oferecida.

Modelo & Método	Parâmetros	MNLI	SST2	MRPC	CoLA	QNLI	QQP	RTE	STSB	Média
RoBERTa _{base} (LoRA _{r=8})	0.30M	87.50	95.10	89.70	63.40	93.30	90.80	86.60	91.50	87.2
RoBERTa _{base} (LoRA + Nosso _{r=8})	0.15M	86.70	93.60	88.60	61.30	92.80	89.70	74.50	90.20	84.70
RoBERTa _{base} (LoRA + Nosso _{r=16})	0.30M	86.92	93.84	88.23	60.14	92.69	90.04	76.30	90.18	84.79

Tabela 6 – Resultados da seleção de conjunto de camadas com nosso método no RoBERTa_{base} com aumento de *rank*. São utilizadas como métricas de avaliação: a correlação de Matthew para o CoLA, a correlação de Pearson para o STSB e a acurácia para as demais tarefas. Produzimos todos os resultados como a média de três execuções.

4.7 Quanto à eficiência computacional

A ausência de módulos LoRA em camadas do modelo durante o ajuste provoca uma diminuição do custo computacional do processo como um todo. Para quantificar esse aumento em eficiência computacional, realizamos medidas de tempo de treino e uso de memória do modelo RoBERTa_{base} no *benchmark* GLUE. Complementando a análise,

realizamos medidas do uso de memória por passo de treino com o modelo LLaMA-2-7B na base de dados MetaMathQA.

A Figura 7 mostra que aplicar a seleção de camadas com nosso método leva a uma diminuição significativa no tempo de treino. A aceleração varia conforme as camadas selecionadas, alcançando um máximo de cerca de 140% e um mínimo de aproximadamente 110%. Essa variação ocorre, pois, além dos cálculos e operações do otimizador que evitamos ao remover módulos de qualquer camada, a remoção dos módulos das primeiras camadas faz o cálculo do gradiente não ser necessário nelas, o que provoca uma aceleração adicional.

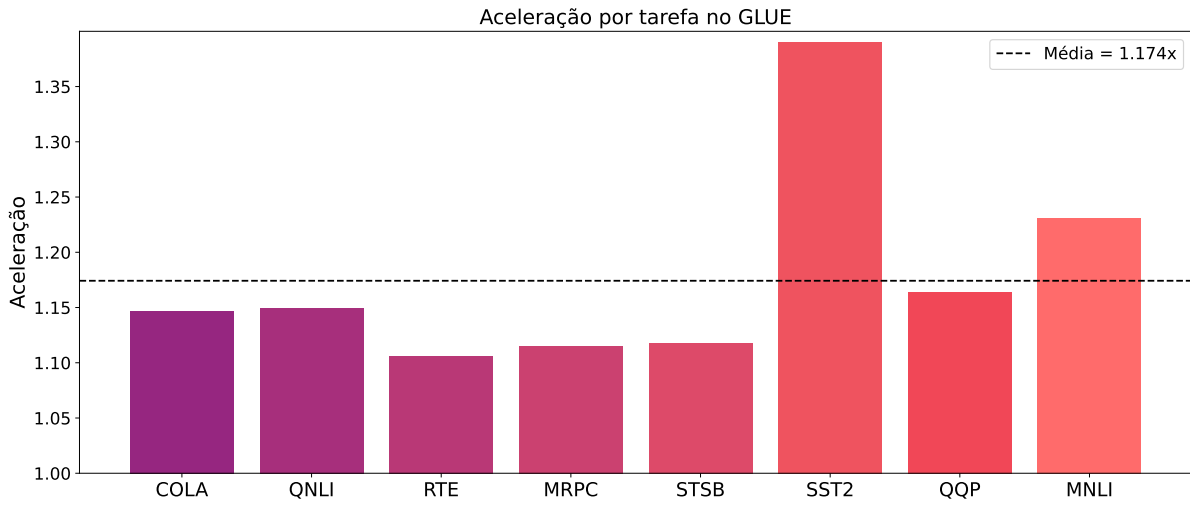


Figura 7 – Aceleração em treino do modelo $\text{RoBERTa}_{\text{base}}$ por tarefa no *benchmark* GLUE. A aceleração é calculada como a razão entre o tempo de execução do ajuste com todos os módulos e o tempo de execução do ajuste com apenas os módulos selecionados. A média é calculada como a média geométrica entre as acelerações das tarefas.

Para estimar o impacto energético dessa aceleração, realizamos a simulação de um cenário hipotético, mas verossímil, utilizando a calculadora de emissão de carbono de Lacoste et al. 2019. Considerando a utilização de 100 horas de uma placa NVIDIA A100 para experimentos envolvendo ajuste fino, o uso de energia seria equivalente à emissão de aproximadamente 22.8 kg de CO_2 . Uma vez que o gasto de energia é proporcional ao tempo de treino, a aceleração média alcançada por nosso método reduziria a emissão de carbono em 17.4%, resultando em uma queda de 3.94 kg de CO_2 emitido. Este resultado, em consonância com os experimentos anteriores, evidencia que o método gera um impacto significativo mesmo com pouca queda de desempenho preditivo.

Da perspectiva de uso de memória, a Figura 8 ilustra a redução que a seleção de camadas com o *CKA* promove. Não só a quantidade de memória alocada, a qual o PyTorch efetivamente ocupa com tensores, mas também a quantidade de memória reservada, que representa toda memória que o *framework* reserva para sua operação, apresentam quedas

significativas. Enquanto em algumas tarefas, como QQP, a redução no uso de memória é de cerca de 11%, em outras, como SST-2, ela chega a aproximadamente 39%. Acreditamos que essa diferença entre configurações de tarefas específicas tem origem na ausência de participação das camadas iniciais não selecionadas no processo de retropropagação para atualização dos pesos, assim como sugerimos na aceleração em treino. Em especial, com $\text{RoBERTa}_{\text{base}}$, hipotetizamos que essa diferença é mais acentuada devido ao modelo apresentar grande parte de seus parâmetros fora de suas camadas. Apesar da queda adicional no uso de memória durante o treino ocorrer apenas em configurações específicas, a redução média em torno de 20% já é significativa.

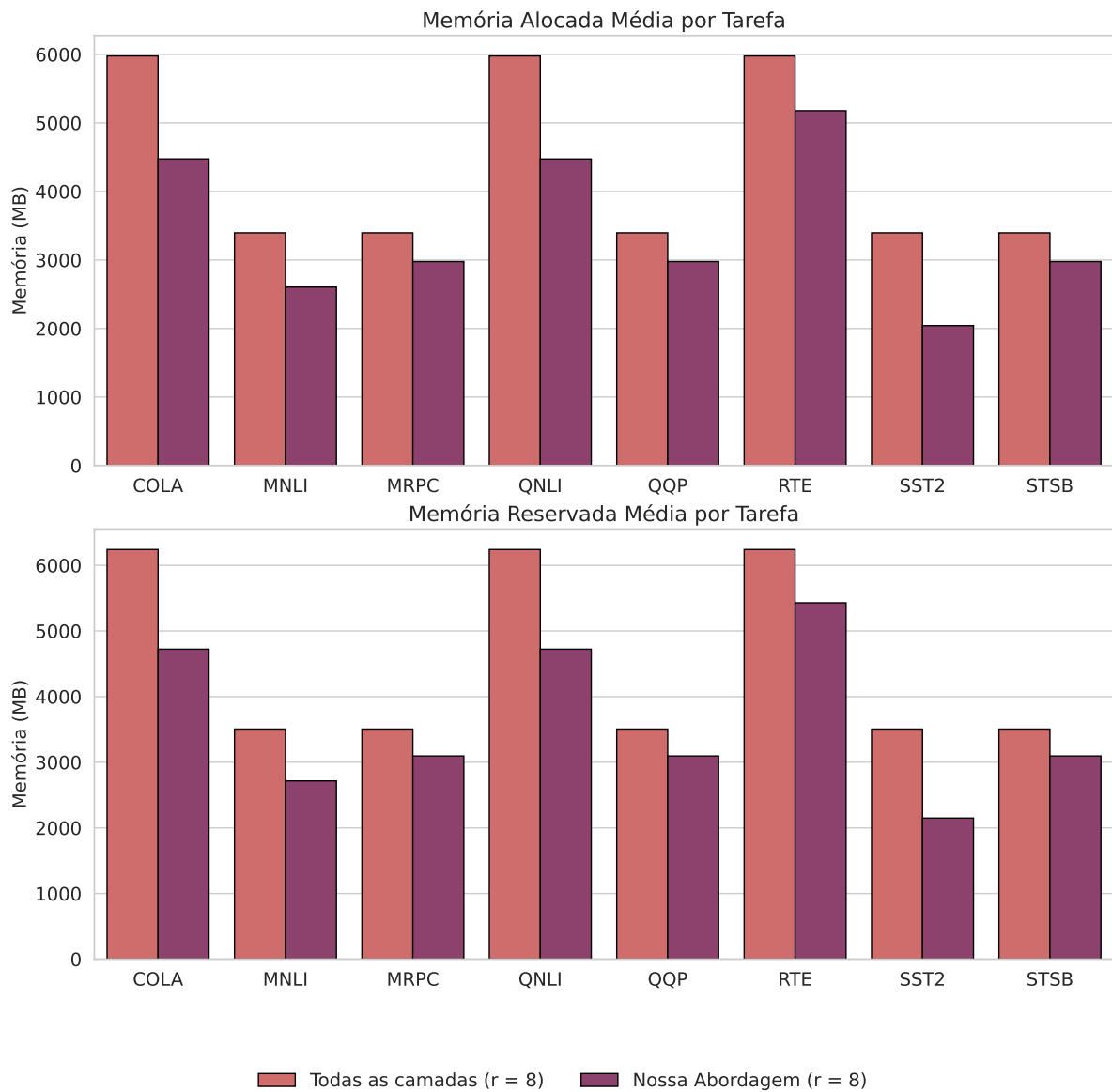


Figura 8 – Memória alocada e reservada da GPU em *megabytes* (MB) por tarefa do modelo $\text{RoBERTa}_{\text{base}}$ no *benchmark* GLUE.

A Figura 9 mostra a redução do uso de memória da GPU ao selecionar metade das camadas para aplicar módulos LoRA com nosso método no modelo *decoder-only* LLaMA-2-7B. Nota-se que este modelo, que possui em seu subconjunto de camadas selecionadas as iniciais, apresenta uma redução média de 15%, maior que o alcançado com RoBERTa_{base} com configurações similares. Este fenômeno reforça a hipótese de que a menor quantidade de parâmetros nas camadas do RoBERTa_{base} diminui o impacto em memória de configurações que incluem as primeiras camadas.

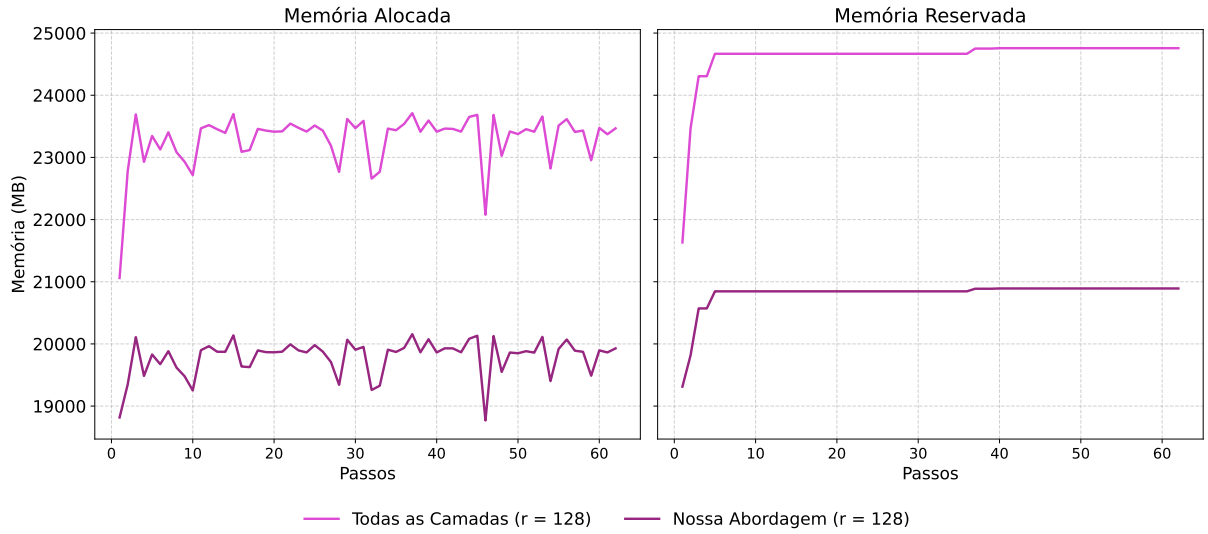


Figura 9 – Memória alocada e reservada da GPU em *megabytes* (MB) por passo de treino do modelo LLaMA-2-7B na base de dados MetaMathQA.

4.8 Efetividade em modelos multimodais

Além do domínio de linguagem, avaliamos a efetividade da seleção de camadas para o ajuste fino utilizando nosso método com o modelo multimodal LLaVA-1.5-7B. Ajustamos-o na base de dados de perguntas e respostas em ciências *ScienceQA* (LU et al., 2022) e avaliamos na sua própria partição de teste. Para assegurar reprodutibilidade e avaliação adequada, utilizamos o *framework* de avaliação *lmms-eval* (LI et al., 2024).

Em concordância com o próprio processo de desenvolvimento do modelo LLaVA, no qual apenas sua estrutura de linguagem é ajustada após um pré-treino de sua camada de projeção, realizamos o ajuste fino apenas no modelo de linguagem. Além do ajuste com nosso método, realizamos o ajuste aplicando módulos LoRA em todas as camadas para comparação.

Com uma acurácia média em três execuções de 79.26%, nosso método apresenta uma queda de desempenho preditivo de apenas 0.23 pontos percentuais em relação ao

ajuste em todas as camadas, que apresenta acurácia média de 79.49%. Esta diferença negligenciável, mesmo com uma redução de 160M para 80M de parâmetros treináveis, indica a seleção de um subconjunto adequado de camadas para o ajuste fino. Tais resultados confirmam a eficácia do método para modelos multimodais, os quais apresentam processos de treino e arquiteturas diferentes dos grandes modelos de linguagem extensivamente abordados anteriormente.

5 Considerações Finais

5.1 Conclusões

Este trabalho explora a seleção de camadas para o ajuste fino como uma direção ortogonal aos métodos baseados no *Low-Rank Adaptation* (LoRA), fornecendo uma alternativa para tornar o treino com estas técnicas ainda mais eficiente do ponto de vista computacional. Com fundamentos experimentais e da literatura, construímos um método centrado nos dados. Atribuímos importância a uma camada com base na sua contribuição para mudanças nas representações internas do modelo.

Nossos extensivos experimentos confirmam a efetividade da seleção de camadas para o ajuste fino com o CKA. Em síntese, em modelos *encoder-only*, alcançamos uma redução de 50% dos parâmetros treináveis com uma queda em média máxima de apenas 2.5 pontos percentuais em tarefas de compreensão de texto. Em modelos *decoder-only*, ao empregar o subconjunto com metade das camadas selecionado com nossa técnica, observamos quedas marginais de performance preditiva ou até melhoras em tarefas relacionadas à capacidade de resolução de problemas de matemática e proficiência em programação. Por fim, em um popular modelo multimodal, obtemos a queda negligenciável de 0.23 pontos percentuais na acurácia em nossa configuração eficiente para tarefa de resposta a perguntas de ciências.

Além das comparações diretas com o ajuste de todas as camadas, exploramos aspectos específicos do nosso método. A comparação com heurísticas simples demonstra que nosso método apresenta um desempenho preditivo superior ou mais consistente em relação a essas formas simples de escolher. Os experimentos tratando da quantidade de blocos selecionados mostram a capacidade do nosso método de adequar-se a situações com diferentes quantidades de recursos computacionais disponíveis. O estudo sobre a redistribuição de parâmetros com o *rank* reafirma a natureza do nosso método de captar interações com características gerais dos dados da tarefa. Por fim, o experimento sobre eficiência computacional quantifica a redução em memória e aceleração em treino que nosso método promove.

Do ponto de vista prático, este trabalho não só contribui para tornar o paradigma de ajuste fino de grandes modelos pré-treinados viável para ambientes de menor capacidade computacional, mas também colabora para o menor gasto energético nesta etapa de treino, alinhando-se com o desenvolvimento sustentável de inteligência artificial. Com respeito ao ajuste em ambientes de recursos limitados, o método desenvolvido mostrou-se efetivo tanto para a redução de 50% dos parâmetros treináveis, a qual abordamos em diversos cenários, quanto para diferentes reduções. Portanto, a estratégia de seleção de camadas

com o CKA demonstrou-se como uma alternativa que se adequa a cenários de diferentes capacidades computacionais, sendo diretamente aplicável, dada a flexibilidade para a escolha da quantidade de camadas selecionadas. Referente ao menor gasto energético, a seleção de um subconjunto de camadas para o ajuste fino aumenta a eficiência energética deste processo. Observamos uma aceleração média de 117.4% com o modelo RoBERTa_{base} no *benchmark* GLUE, o que se converte em reduções proporcionais no uso de energia.

Do ponto de vista teórico, esta pesquisa trata a seleção de camadas para o ajuste fino de uma perspectiva diferente da existente na literatura. Em contraste com o trabalho desenvolvido por Lodha et al. 2023, abordamos diferentes arquiteturas, incluindo modelos *decoder-only* e multimodais, tarefas complexas, uso concomitante a outras técnicas de PEFT, como LoRA e suas variantes, e um índice de importância para as camadas baseado na dissimilaridade de representações internas. Nesse sentido, nosso trabalho não apenas estende a seleção de camadas para o ajuste fino para modelos e tarefas modernas, como também introduz o uso de métricas de similaridade para resolução deste problema.

5.2 Perspectivas de continuidade

Este trabalho compreende a seleção de camadas para o ajuste fino com o uso do CKA. Apesar dos resultados positivos, existem direções promissoras de pesquisa que, ainda que alinhadas com o escopo do estudo, não puderam ser incluídas. Primeiro, é possível explorar a importância de uma camada de outras óticas além da similaridade das representações internas. Por exemplo, trabalhos recentes ressaltam a importância de abordagens ligadas à geometria e informação das representações (SKEAN et al., 2025). Além disso, este trabalho aborda a aplicação de módulos LoRA em um subconjunto de camadas consideradas mais importantes para uma tarefa, outra abordagem para aumentar a eficiência computacional seria a definição do *rank* ideal de cada camada. Por fim, nosso método captura de forma isolada a contribuição de uma camada para as mudanças nas representações internas do modelo. Uma possível extensão é a adição de formas de abordar as interações entre diferentes camadas, para explorar possíveis dependências, redundâncias ou sinergias durante o processo de treino.

Referências

- AUSTIN, J. et al. Program synthesis with large language models. In: *arXiv*. [S.l.: s.n.], 2021. Citado na página 27.
- BENGIO, Y.; AL. et. *International AI Safety Report*. 2025. Citado na página 12.
- CHEN, e. a. M. Evaluating large language models trained on code. In: *arXiv*. [S.l.: s.n.], 2021. Citado na página 27.
- COBBE, K. et al. *Training Verifiers to Solve Math Word Problems*. 2021. Citado na página 27.
- DEEPSEEK-AI et al. Deepseek-v3 technical report. *Computing Research Repository (CoRR)*, 2024. Citado na página 12.
- DETTMERS, T. et al. Qlora: Efficient finetuning of quantized llms. In: *Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2023. Citado na página 12.
- DING, N. et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence (Nat. Mach. Intell.)*, 2023. Citado 2 vezes nas páginas 12 e 19.
- FAN, C. et al. Make loRA great again: Boosting loRA with adaptive singular values and mixture-of-experts optimization alignment. In: *International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2025. Citado na página 13.
- GEVA, M. et al. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [S.l.: s.n.], 2022. Citado na página 14.
- GRETTON, A. et al. Measuring statistical dependence with hilbert-schmidt norms. In: *Algorithmic Learning Theory (ALT)*. [S.l.: s.n.], 2005. Citado na página 21.
- HAN, Z. et al. Parameter-efficient fine-tuning for large models: A comprehensive survey. *Transactions on Machine Learning Research (TMLR)*, 2024. Citado na página 12.
- HE, P. et al. Deberta: Decoding-enhanced bert with disentangled attention. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2021. Citado 2 vezes nas páginas 19 e 25.
- HENDRYCKS, D. et al. Measuring mathematical problem solving with the MATH dataset. In: *Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2021. Citado na página 27.
- HU, E. J. et al. Lora: Low-rank adaptation of large language models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2022. Citado 6 vezes nas páginas 12, 20, 25, 26, 28 e 29.
- HUANG, Q. et al. HiRA: Parameter-efficient hadamard high-rank adaptation for large language models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2025. Citado 2 vezes nas páginas 13 e 31.

- JIANG, A. Q. et al. Mistral 7b. In: *arXiv*. [S.l.: s.n.], 2023. Citado 2 vezes nas páginas 19 e 27.
- JIN, M. et al. *Exploring Concept Depth: How Large Language Models Acquire Knowledge and Concept at Different Layers?* 2025. Citado 2 vezes nas páginas 29 e 31.
- KLABUNDE, M. et al. Resi: A comprehensive benchmark for representational similarity measures. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2025. Citado na página 23.
- KORNBLITH, S. et al. Similarity of neural network representations revisited. In: *International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 13 e 21.
- LACOSTE, A. et al. Quantifying the carbon emissions of machine learning. *arXiv*, 2019. Citado na página 33.
- LEE, Y. et al. Surgical fine-tuning improves adaptation to distribution shifts. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2023. Citado na página 22.
- LI, B. et al. *LMMs-Eval: Accelerating the Development of Large Multimodal Models*. Zenodo, 2024. Disponível em: <<https://github.com/EvolvingLMMs-Lab/lmms-eval>>. Citado na página 35.
- LIU, H. et al. Visual instruction tuning. In: *Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2023. Citado 2 vezes nas páginas 19 e 20.
- LIU, Y. et al. Roberta: A robustly optimized bert pretraining approach. In: *arXiv*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 19 e 25.
- LODHA, A. et al. On surgical fine-tuning for language encoders. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [S.l.: s.n.], 2023. Citado 3 vezes nas páginas 22, 26 e 38.
- LU, P. et al. Learn to explain: Multimodal reasoning via thought chains for science question answering. In: *Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2022. Citado na página 35.
- MASLEJ, N. *Artificial intelligence index report*. [S.l.]: The Stanford for Human-Centered AI (HAI), 2025. <<https://hai.stanford.edu/ai-index/2025-ai-index-report>>. Citado na página 12.
- MENG, F.; WANG, Z.; ZHANG, M. PiSSA: Principal singular values and singular vectors adaptation of large language models. In: *Conference on Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2024. Citado 3 vezes nas páginas 21, 26 e 28.
- OPENAI. GPT-4 technical report. *Computing Research Repository (CoRR)*, 2023. Citado na página 12.
- SKEAN, O. et al. Layer by layer: Uncovering hidden representations in language models. In: *International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2025. Citado 2 vezes nas páginas 30 e 38.

- TEAM, G. et al. Gemma: Open models based on gemini research and technology. In: *arXiv*. [S.l.: s.n.], 2024. Citado 2 vezes nas páginas 19 e 27.
- TIAN, C. et al. Hydralora: An asymmetric lora architecture for efficient fine-tuning. In: *Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2024. Citado na página 13.
- TOUVRON, H. et al. Llama 2: Open foundation and fine-tuned chat models. In: *arXiv*. [S.l.: s.n.], 2023. Citado 3 vezes nas páginas 12, 19 e 27.
- VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2017. Citado 3 vezes nas páginas 12, 15 e 16.
- WANG, A. et al. Glue: A multi-task benchmark and analysis platform for natural language understanding. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 25 e 28.
- YU, L. et al. Metamath: Bootstrap your own mathematical questions for large language models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2024. Citado na página 27.
- ZHANG, Q. et al. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2023. Citado 2 vezes nas páginas 13 e 14.
- ZHENG, T. et al. OpenCodeInterpreter: Integrating code generation with execution and refinement. In: *Findings of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2024. Citado na página 27.

Apêndices

APÊNDICE A – A primeira camada de modelos *encoder-only*

Observamos que nosso método apresenta valores contraintuitivos para a primeira camada dos modelos *encoder-only*. A Figura 10 mostra que a primeira camada consistentemente apresenta similaridade nula entre sua entrada e saída, o que implica em uma importância máxima.

Este fenômeno ocorre devido ao CKA utilizar matrizes centralizadas no seu cálculo. Por analisarmos o token $\langle \text{CLS} \rangle$ de cada camada, na entrada da primeira camada em que não há ainda transformações, a matriz centralizada torna-se nula, pois a média é o próprio $\langle \text{CLS} \rangle$ presente em todas as amostras. Dessa forma, o CKA torna-se nulo também.

Além do aspecto relacionado à métrica, investigamos experimentalmente o uso da primeira camada nos conjuntos de camadas selecionadas. Ao comparar o desempenho preditivo do modelo $\text{RoBERTa}_{\text{base}}$ ajustado com e sem a primeira camada como parte do conjunto de camadas, observamos que a diferença média é de menos de 0.2 pontos percentuais entre as duas configurações. Concluimos, então, que a primeira camada não atua de forma primordial para todas as tarefas, sugerindo que a métrica não reflete a importância da camada. Portanto, para garantir consistência e evitar a não seleção de camadas possivelmente mais importantes, excluimos a primeira camada do conjunto de camadas selecionadas em modelos *encoder-only*.

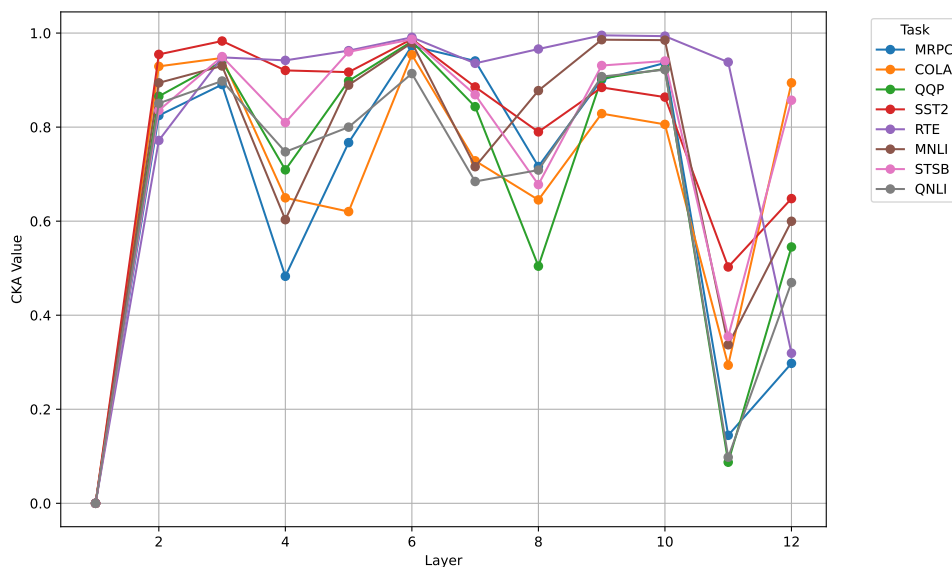


Figura 10 – Valor do CKA por camada do modelo $\text{RoBERTa}_{\text{base}}$ em diferentes tarefas do *benchmark* GLUE.